

Resumen de Funciones de Haskell

Guía rápida para el parcial — descripción breve y ejemplo simple de cada función

■ Algunas de estas funciones no están en el Preludio, sino en los módulos **Data.List** y/o **Data.Maybe**. Para usarlas hay que incluir `import Data.List` y/o `import Data.Maybe` al inicio del archivo fuente.

Funciones de orden superior sobre listas

Función	Qué hace	Ejemplo
foldr	Pliega la lista de derecha a izquierda: $f\ x1\ (f\ x2\ (\dots\ z))$	<code>foldr (+) 0 [1,2,3] = 6</code>
foldl	Pliega la lista de izquierda a derecha: $f\ (\dots(f\ z\ x1)\ x2\ \dots)$	<code>foldl (+) 0 [1,2,3] = 6</code>
foldr1	Como foldr pero usa el último elemento como caso base (sin z)	<code>foldr1 (+) [1,2,3] = 6</code>
foldl1	Como foldl pero usa el primer elemento como caso base (sin z)	<code>foldl1 (-) [10,1,2] = 7</code>
map	Aplica una función a cada elemento de la lista	<code>map (*2) [1,2,3] = [2,4,6]</code>
zipWith	Combina dos listas elemento a elemento usando una función	<code>zipWith (+) [1,2] [10,20] = [11,22]</code>
filter	Se queda con los elementos que cumplen un predicado	<code>filter even [1,2,3,4] = [2,4]</code>
concatMap	Aplica una función que devuelve listas y concatena el resultado	<code>concatMap (\x -> [x,x]) [1,2] = [1,1,2,2]</code>
concat	Concatena una lista de listas en una sola	<code>concat [[1,2],[3],[4,5]] = [1,2,3,4,5]</code>

Predicados y funciones booleanas

Función	Qué hace	Ejemplo
all	True si TODOS los elementos cumplen el predicado	<code>all even [2,4,6] = True</code>
any	True si ALGÚN elemento cumple el predicado	<code>any odd [2,4,6] = False</code>
and	True si todos los booleanos de la lista son True	<code>and [True,True] = True</code>
or	True si algún booleano de la lista es True	<code>or [False,True] = True</code>
null	True si la lista/estructura está vacía	<code>null [] = True</code>
not	Niega un booleano	<code>not True = False</code>
odd	True si el número es impar	<code>odd 3 = True</code>
even	True si el número es par	<code>even 4 = True</code>

Aritmética

Función	Qué hace	Ejemplo
mod	Resto de la división entera (módulo)	<code>7 `mod` 3 = 1</code>

Función	Qué hace	Ejemplo
div	Cociente de la división entera	<code>7 `div` 2 = 3</code>

Comparación y ordenamiento

Función	Qué hace	Ejemplo
(==)	Igualdad entre dos valores	<code>3 == 3 = True</code>
(/=)	Desigualdad entre dos valores	<code>3 /= 4 = True</code>
compare	Compara dos valores y devuelve LT, EQ o GT	<code>compare 2 5 = LT</code>
comparing	Crea una función de comparación a partir de una función clave (Data.Ord)	<code>sortBy (comparing length) [[1,2],[1]] = [[1],[1,2]]</code>
sort	Ordena una lista de menor a mayor (Data.List)	<code>sort [3,1,2] = [1,2,3]</code>
sortBy	Ordena una lista usando una función de comparación (Data.List)	<code>sortBy compare [3,1,2] = [1,2,3]</code>
max	Devuelve el mayor entre dos valores	<code>max 3 5 = 5</code>
maximum	Devuelve el mayor elemento de una lista	<code>maximum [3,1,5] = 5</code>
maximumBy	Máximo según una función de comparación (Data.List)	<code>maximumBy (comparing length) [[1],[1,2]] = [1,2]</code>
min	Devuelve el menor entre dos valores	<code>min 3 5 = 3</code>
minimum	Devuelve el menor elemento de una lista	<code>minimum [3,1,5] = 1</code>
minimumBy	Mínimo según una función de comparación (Data.List)	<code>minimumBy (comparing length) [[1],[1,2]] = [1]</code>

Construcción y acceso a listas

Función	Qué hace	Ejemplo
(++)	Concatena dos listas	<code>[1,2] ++ [3,4] = [1,2,3,4]</code>
head	Devuelve el primer elemento de la lista	<code>head [1,2,3] = 1</code>
tail	Devuelve la lista sin el primer elemento	<code>tail [1,2,3] = [2,3]</code>
init	Devuelve la lista sin el último elemento	<code>init [1,2,3] = [1,2]</code>
last	Devuelve el último elemento de la lista	<code>last [1,2,3] = 3</code>
length	Devuelve la cantidad de elementos	<code>length [1,2,3] = 3</code>
reverse	Invierte el orden de la lista	<code>reverse [1,2,3] = [3,2,1]</code>
replicate	Crea una lista repitiendo un valor n veces	<code>replicate 3 'a' = "aaa"</code>
repeat	Crea una lista infinita repitiendo un valor	<code>take 3 (repeat 5) = [5,5,5]</code>
iterate	Lista infinita aplicando una función repetidamente	<code>take 4 (iterate (*2) 1) = [1,2,4,8]</code>
take	Toma los primeros n elementos	<code>take 2 [1,2,3] = [1,2]</code>
drop	Descarta los primeros n elementos	<code>drop 2 [1,2,3] = [3]</code>

Función	Qué hace	Ejemplo
takeWhile	Toma elementos mientras se cumpla el predicado	<code>takeWhile even [2,4,5,6] = [2,4]</code>
dropWhile	Descarta elementos mientras se cumpla el predicado	<code>dropWhile even [2,4,5,6] = [5,6]</code>
span	Divide la lista en (takeWhile p, dropWhile p)	<code>span even [2,4,5,6] = ([2,4],[5,6])</code>
nub	Elimina elementos duplicados (Data.List)	<code>nub [1,1,2,3,3] = [1,2,3]</code>
union	Unión de dos listas sin duplicados (Data.List)	<code>union [1,2] [2,3] = [1,2,3]</code>
intersect	Intersección de dos listas (Data.List)	<code>intersect [1,2,3] [2,3,4] = [2,3]</code>

Búsqueda dentro de listas

Función	Qué hace	Ejemplo
elem	True si el elemento está en la lista	<code>3 `elem` [1,2,3] = True</code>
find	Devuelve el primer elemento que cumple el predicado, envuelto en Maybe (Data.List)	<code>find even [1,3,4,5] = Just 4</code>
lookup	Busca un valor asociado a una clave en una lista de pares	<code>lookup 2 [(1,"a"),(2,"b")] = Just "b"</code>

Data.Maybe

Función	Qué hace	Ejemplo
isNothing	True si el valor es Nothing	<code>isNothing Nothing = True</code>
isJust	True si el valor es Just algo	<code>isJust (Just 5) = True</code>
fromJust	Extrae el valor de un Just (falla con Nothing)	<code>fromJust (Just 5) = 5</code>
maybe	Aplica una función si es Just, o devuelve un default si es Nothing	<code>maybe 0 (+1) (Just 5) = 6</code>

Mónadas

Función	Qué hace	Ejemplo
(>>=)	Bind monádico: encadena una acción con una función que produce otra acción	<code>[1,2] >>= (\x -> [x,x]) = [1,1,2,2]</code>

Data.Char

Función	Qué hace	Ejemplo
ord	Convierte un carácter a su código numérico (Data.Char)	<code>ord 'a' = 97</code>
chr	Convierte un código numérico a su carácter (Data.Char)	<code>chr 97 = 'a'</code>

import Data.List: nub, sort, sortBy, union, intersect, find, maximumBy, minimumBy | **import Data.Maybe:** isNothing, isJust, fromJust, maybe | **import Data.Ord:** comparing | **import Data.Char:** ord, chr